Representation-Regularized World Models for Visual Locomotion Transfer: An R2-Dreamer `walker_walk` → `walker_run` Study on DMC Vision

Thomas Georges*

June 16, 2026

Abstract

We study transfer in a reconstruction-free, representation-regularized DreamerV3 variant ("R2-Dreamer" [11, 1]) on the DeepMind Control Suite [8] from pixels. A 25M-parameter world model is trained on `walker_walk` and then either fine-tuned on `walker_run` from the walk checkpoint, or trained on `walker_run` from scratch, under an identical 400k-step target budget. The source policy effectively solves the walk task (eval return 968). On the run task, fine-tuning reaches 742 versus 563 from scratch, a +179 (+32%) advantage at equal budget, with a much larger head start earlier: the warm-started policy scores ~ 375 before any run-specific training (vs. ~ 25 for scratch) and leads by roughly $4\times$ at the 100k-step mark, with the gap peaking near +448 and then narrowing as the scratch baseline—still rising at the budget limit—catches up. Training is stable for all three runs, and the R2 redundancy-reduction objective converges cleanly. A PCA analysis of the recurrent latent states shows that the two same-task policies occupy clearly separated regions of feature space and that the fine-tuned model traces a larger, cyclically structured (limit-cycle-like) latent manifold, while the scratch model collapses into a compact blob—so fine-tuning changes not only learning speed but the learned representation itself. Results are single-seed and at a fixed budget; we discuss the resulting limits on the conclusions.

1 Objective

The goal of this track is a faithful, budget-feasible demonstration of the canonical Dreamer transfer experiment—train a latent world model on a source task, save and reload it, adapt it to a shifted task, and compare fine-tuning against training from scratch—using the *representation-regularized* R2-Dreamer objective on image observations. Concretely we run the three-way comparison

$$\begin{aligned} \texttt{source\_base}: &\ \texttt{walker\_walk} \text{ (from scratch)},\\ \texttt{target\_finetune}: &\ \texttt{walker\_run} \text{ (initialized from } \texttt{source\_base}),\\ \texttt{target\_scratch}: &\ \texttt{walker\_run} \text{ (from scratch)}, \end{aligned}$$

all on `dmc_vision` (64×64 RGB), orchestrated from a single Colab notebook against an unmodified upstream checkout,[1] with all checkpoints, metrics, videos, and figures persisted to Google Drive.

*Engineering and experiment automation developed with Claude (Anthropic) in the `World_Models_LAS` repository.

[1] `NM512/r2dreamer` [11], itself a PyTorch reimplementation of DreamerV3 [1] extending `NM512/dreamerv3-torch` [12]; behavioural changes are applied as marker-guarded, idempotent, reversible textual patches at launch time (checkpoint load/save, interval checkpoints, headless-render guard, serial-env fallback).

Every number and figure in this report is read directly from the committed outputs of the results notebook (`notebooks/06_r2dreamer_results.ipynb`).

2 Model

Backbone. The agent is a DreamerV3-style latent world model [1, 2, 3]. A convolutional encoder maps each $64 \times 64 \times 3$ frame to an embedding; a Recurrent State-Space Model (RSSM) [4] carries a deterministic recurrent state and a stochastic latent and predicts forward dynamics; reward and episode-continuation heads decode task quantities from the latent; and an actor–critic is trained entirely "in imagination" on rollouts of the learned dynamics rather than on environment frames [3]. The observation here is vision only (the encoder receives {`image:(64,64,3)`} and no proprioceptive MLP input).

Representation objective (the "R2" in R2-Dreamer). The upstream code exposes a swappable representation loss (`model.rep_loss` $\in$ {`dreamer`, `r2dreamer`, `infonce`, `dreamerpro`}). The standard `dreamer` setting learns the latent through pixel reconstruction with a decoder. We use `rep_loss=r2dreamer`: a *reconstruction-free* objective that learns the representation through a projector head trained with a redundancy-reduction (Barlow-Twins-style) loss on projected latents [6], placing it in the same reconstruction-free model-based family as DreamerPro [5] and contrastive CPC/InfoNCE [7] alternatives. A practical consequence is that the `r2dreamer` model has *no decoder*: there is no imagined-frame "`video_pred`" path (which the upstream agent exposes only for `rep_loss=dreamer`). Visual analysis therefore uses environment-rendered policy rollouts and the geometry of the RSSM latent states, not decoded reconstructions.

Parameter budget. The nominal `size25M` configuration instantiates 26.53M trainable parameters, dominated by the RSSM and the R2 projector head:

Component	Parameters	Role
RSSM	13,442,304	latent dynamics (recurrent + stochastic state)
Projector	5,898,240	R2 representation head (redundancy reduction)
Value (critic)	1,869,951	imagined-return value
Actor (policy)	1,776,396	imagination policy
Reward predictor	1,573,503	reward head
Continue predictor	1,475,713	episode-continuation head
Encoder (CNN)	493,824	$64 \times 64 \times 3 \rightarrow$ embedding
Total	**26,529,931**	

Table 1: World-model components and parameter counts, read from the rollout/ extraction logs. Each saved checkpoint is 325.9 MB (fp32 weights plus Adam optimizer moments); the six most recent interval checkpoints are retained per run (`checkpoint_keep=6`).

3 Experimental setup and resources

Tasks and protocol. Source task `dmc_walker_walk`, target task `dmc_walker_run`, both rendered as 64×64 RGB (Figure 1). The fine-tune run initializes *all* world-model weights from the source checkpoint with a strict state-dict load and does *not* carry over optimizer state

(`pretrained_strict=true`, `load_optimizer=false`); the scratch run is identical except for random initialization. Both target runs share the same 400k-step budget so the comparison is at equal target experience.

Setting	Value	Setting	Value
Observation	`dmc_vision` 64×64×3	Train ratio	224
Model	`size25M`	Parallel envs	8
Representation loss	`r2dreamer` (Barlow-style)	Eval cadence	every 50k steps, 5 episodes
Batch	16 sequences × length 64	Seed	0
Source steps	800k (reached ∼796k)	Target steps	400k (reached ∼396k)

Table 2: Training configuration (the `a100_r2_vision25m` three-way preset in `configs/r2dreamer/`).

Compute and software. Training and post-hoc analysis ran on a single NVIDIA A100 (Google Colab). The stack is PyTorch with Hydra configuration (`NM512/r2dreamer` [11] over `dreamerv3-torch` [12]), DeepMind Control / `dm_control` tasks [8, 9] on the MuJoCo physics engine [10], on a Python 3.11 runtime (the version upstream declares). Image observations are produced by headless CPU rendering through OSMesa (`MUJOCO_GL=osmesa`) while the policy runs on CUDA; a render sanity-check confirms non-blank frames before any rollout (Figure 1). The full run writes model-only interval checkpoints at evaluation boundaries; the source run in this report was interrupted and resumed (three appended metric segments, 326 rows discarded), and the analysis uses the final continuous segment to ∼796k steps.

Figure 1: A representative 64 × 64 RGB observation from the patched DMC vision environment (render sanity-check: non-blank, pixel mean 68, std 33). The policies learn locomotion from frames like this; because `rep_loss=r2dreamer` is decoder-free, the model never reconstructs them.

4 Results

Run	Init	Task	Final eval	Final train	Env steps
`source_base`	random	`walker_walk`	968.3	964.2	∼796k
`target_finetune`	`source_base`	`walker_run`	742.3	693.4	∼396k
`target_scratch`	random	`walker_run`	563.1	529.7	∼396k

Table 3: Final results (DMC episodic return, 0–1000). "Eval" is the deterministic 5-episode evaluation score; "train" is the rolling training return. For all three runs the best eval score equals the final eval score, i.e. no late-training collapse.

4.1 Source task: `walker_walk` is effectively solved

The source model reaches an evaluation return of 968.3 on `walker_walk` (Table 3; the DMC walk reward saturates near 1000). This is the prerequisite for a meaningful transfer test: the checkpoint we fine-tune from is a competent walker, not a half-trained one. Figure 2 shows the monotone climb to that ceiling.

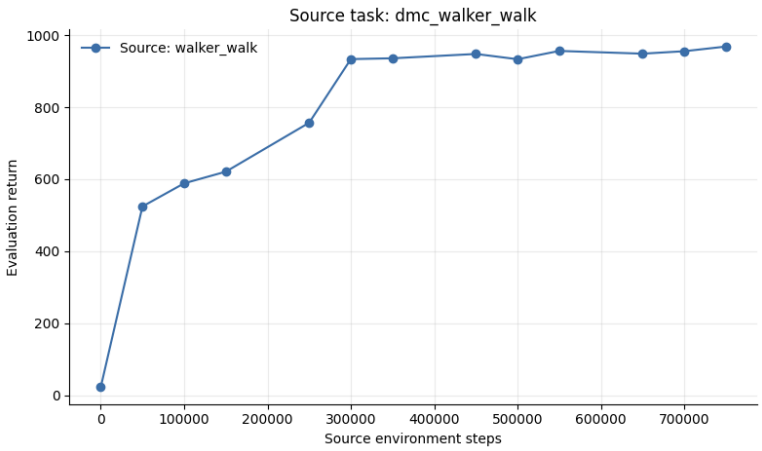

Figure 2: Source-task evaluation return over ∼796k environment steps. The `walker_walk` model converges to ≈ 968, effectively solving the task and providing the initialization for the fine-tune run.

4.2 Transfer: fine-tuning dominates scratch at every budget

This is the central result (Figure 3). Three features stand out. *(i) A large warm start:* at the first evaluation, before any run-specific gradient steps, the fine-tuned policy already returns ∼375 on `walker_run`, versus ∼25 for the random scratch policy—walk and run share enough locomotion structure that the source model transfers almost zero-shot. *(ii) Faster convergence:* by 100k target steps the fine-tune is at ∼600 while scratch is near ∼150, a ∼4× gap; the fine-tune passes 700 by ∼300k. *(iii) A persistent but shrinking lead:* the fine-tune ends at 742 and scratch at 563, a +179 (+32%) advantage at the shared 400k budget.

The advantage curve makes the dynamics explicit: transfer buys the most *early* (peak gap near 100–150k steps), and the steady decline thereafter indicates that the result demonstrates *sample efficiency*, not necessarily a higher asymptotic ceiling—the scratch curve had not flattened at 400k (Section 6).

4.3 Training dynamics and optimization health

The training-episode curves (Figure 4) corroborate the evaluation story and reveal a brief adaptation transient: the fine-tune begins around ∼400 (the inherited walk competence), *dips* to ∼270 as the walk gait is repurposed for running—a short, recoverable negative-transfer phase—then climbs steadily past the scratch run for the remainder of training. The scratch run rises roughly linearly from ∼30 and is still ascending at the budget limit.

Optimization is stable for all three runs (Figure 5): the total optimizer loss and the dynamics/KL, representation, value, and policy terms all descend and remain bounded, with no diver-

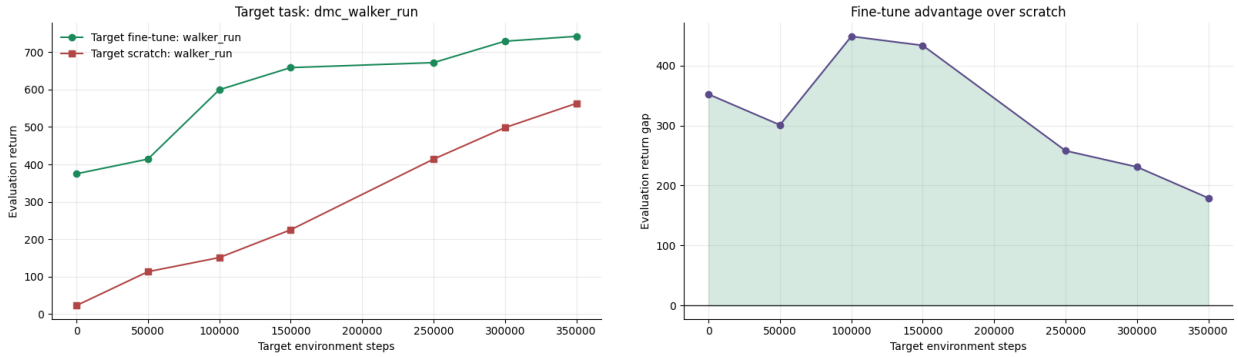

Figure 3: Target-task transfer. **Left:** evaluation return on `walker_run` for fine-tune (green) vs. scratch (red); fine-tune leads at every evaluation point. **Right:** the step-aligned advantage (fine-tune − scratch), which starts at ∼352, peaks at ∼448 around 100k steps, and then declines steadily to +179 at the budget limit as the scratch baseline—still rising and not yet plateaued—closes part of the gap.

gence. In particular the R2-specific Barlow redundancy-reduction term drops sharply and flattens, indicating the reconstruction-free representation objective trained as intended.

4.4 Latent geometry: the two policies live in different basins

We extracted RSSM recurrent features along three evaluation episodes (1500 states each) for the fine-tune and scratch `walker_run` policies and projected them with PCA (Figure 6). Although both policies solve the *same* task, their latent states occupy clearly separated regions—near-linearly separable along the first principal component—so the warm-started model encodes the task differently from the scratch model rather than merely reaching the same representation faster. The 3D view is more striking: the fine-tuned model's latents trace a large, organized ring/limit-cycle manifold consistent with a periodic running gait, whereas the scratch model's latents collapse into a small, compact cluster, mirroring its lower return and less dynamic behaviour.

4.5 Rollout videos

The results notebook renders one camera rollout per checkpoint (`source_base`, `target_finetune`, `target_scratch`) and a rotating 3D latent-PCA animation, all embedded inline. Because the `r2dreamer` model is decoder-free, these are true environment renderings of the executed policy, not imagined reconstructions. Their qualitative content tracks the scores in Table 3: a near-solved walk for the source, a competent run for the fine-tune, and a weaker run for scratch. (A minor corroborating detail: the smoothest, most periodic motion—the source walk—produces the smallest encoded MP4.)

5 Discussion

What the experiment shows. On a visual locomotion shift, a representation-regularized DreamerV3 world model transfers strongly: a competent `walker_walk` model gives an almost zero-shot head start on `walker_run` (∼ 375 vs. ∼ 25), learns several times faster mid-training (∼ 4× at 100k steps), and retains a clear +179 advantage at a fixed 400k budget. The whole machinery—strict checkpoint save/reload across tasks, the reconstruction-free R2 objective, and

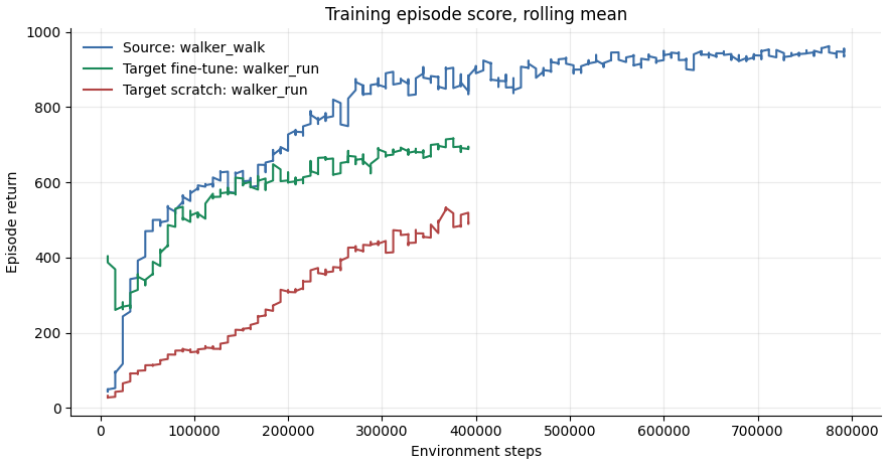

Figure 4: Rolling-mean training episode return (window 8). Source `walker_walk` (blue) climbs to $\sim$950 over 800k steps; the `walker_run` fine-tune (green) starts high, dips during gait re-adaptation, and converges near $\sim$690; scratch (red) rises from near zero to $\sim$530 and is still increasing.

headless vision rendering—works end to end, and the comparison is clean because source, fine-tune, and scratch differ only in initialization.

Efficiency versus ceiling. The shrinking advantage curve (Figure 3, right) is the honest qualifier: scratch was still improving at the budget limit, so the evidence supports a large *sample-efficiency* benefit from transfer rather than a proven higher ceiling. Whether the +179 gap is permanent or merely a head start that a much longer scratch run would erase is not resolved by this budget.

Representation, not just speed. The latent-geometry result is the most interesting qualitative finding. Two policies trained to the same task from different initializations end up in measurably different, near-linearly separable representational regions, and the better policy carries a richer, cyclically structured latent manifold. This suggests fine-tuning lands the world model in a distinct basin shaped by the source task—useful context for interpreting transfer beyond the scalar return, and a reminder that the reconstruction-free R2 latent is structured enough to read locomotion phase from directly.

Why R2 / reconstruction-free here. Replacing the pixel decoder with a redundancy-reduction objective [6, 5] keeps the model focused on control-relevant latent structure and removes the cost and artifacts of pixel reconstruction; the trade-off is the loss of decoded "imagination" visualizations. The clean convergence of the Barlow term and the strong control performance indicate the objective is adequate for this task, though this report does not isolate its contribution against the standard `dreamer` reconstruction loss.

6 Threats to validity and caveats

- **Single seed.** All three runs are seed 0; there are no error bars. The mid-training $\sim 4\times$ gap and the +179 final advantage are large relative to typical DMC run-to-run noise, but

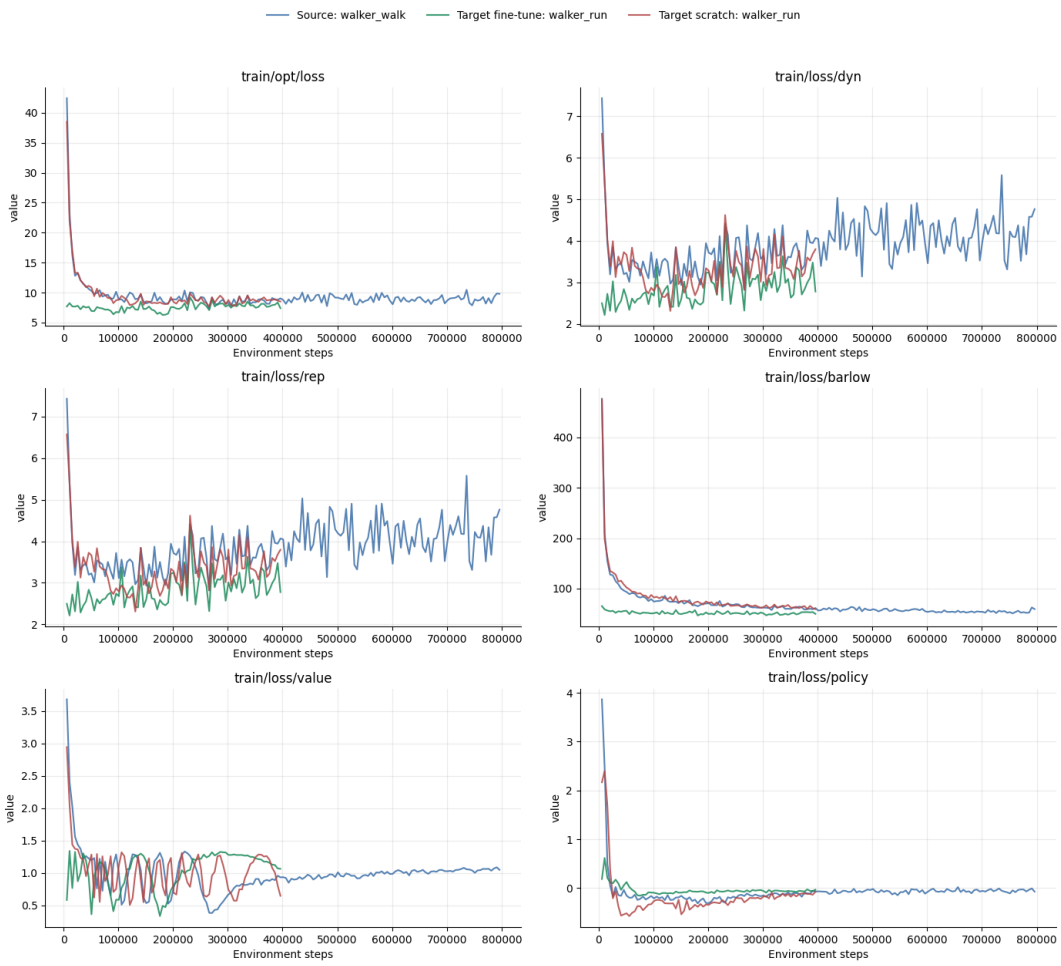

Figure 5: Optimization diagnostics across the three runs (source blue, fine-tune green, scratch red). All loss components are bounded and convergent; the `train/loss/barlow` panel (the R2 redundancy-reduction term) converges cleanly. The source run is noisier at high step counts but does not diverge.

statistical claims would require additional seeds.

- **Fixed budget, scratch not converged.** The scratch baseline is still rising at 400k steps, so the advantage measures sample efficiency at a fixed budget, not an asymptotic ceiling difference.
- **Coarse evaluation.** Evaluation uses 5 deterministic episodes every 50k steps; the curves are therefore mildly noisy and the reported scores are point estimates.
- **PCA variance.** The latent projections explain only $\sim 17\%$ of the variance in three components; the separation and ring structure are real on the dominant axes but should not be read as the full latent geometry.
- **No representation-loss ablation.** We run only `rep_loss=r2dreamer`; we do not compare against the `dreamer`, `infonce`, or `dreamerpro` objectives on this task, so the report characterizes *this* configuration rather than attributing the result to the R2 objective specifically.

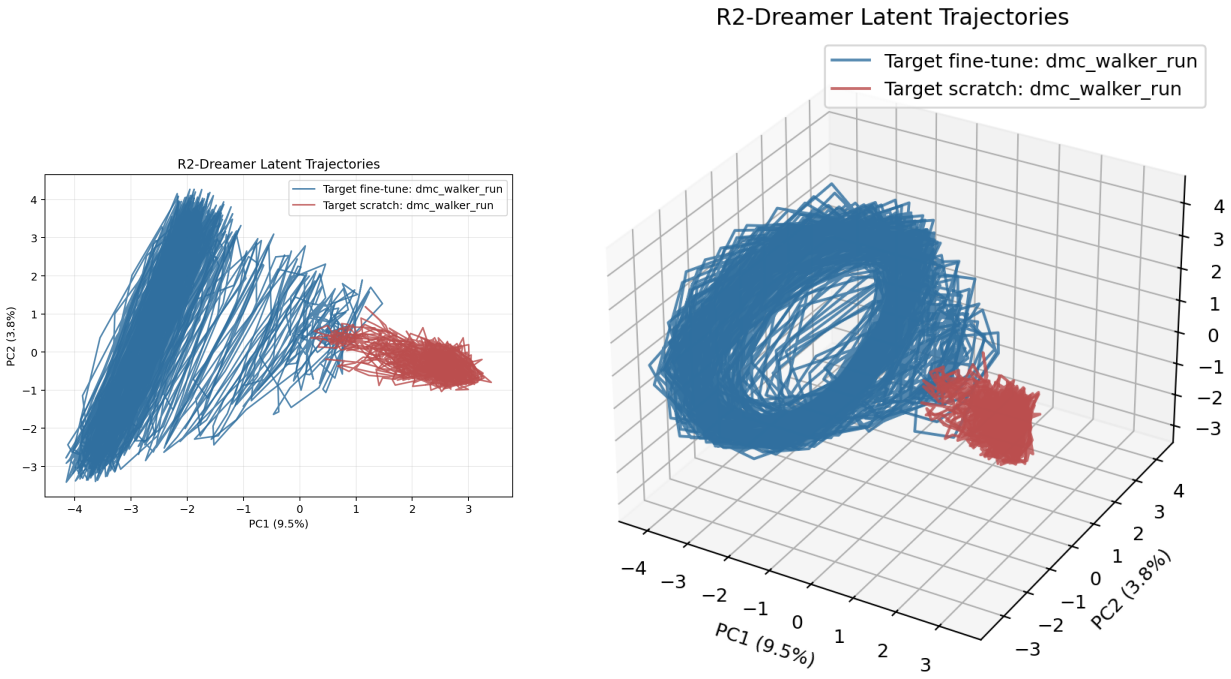

Figure 6: Shared-basis PCA of RSSM latent trajectories, fine-tune (blue) vs. scratch (red) on `walker_run`. **Left (2D):** the two policies separate along PC1. **Right (3D):** the fine-tune traces a large, cyclically structured manifold; the scratch policy occupies a compact blob. The top three principal components explain only $\approx$9.5/3.8/3.5% of the variance ($\sim$17% total), so these projections capture the dominant linear structure of a high-dimensional latent space, not its entirety (Section 6).

- **Decoder-free.** With `rep_loss=r2dreamer` there is no decoder, so no reconstruction or imagination-quality metric is available; visual evidence is limited to executed-policy rollouts and latent geometry.
- **Interrupted source run.** The source run was resumed (three appended metric segments); the analysis uses the final continuous segment. The reloaded checkpoint and all transfer results are unaffected.

7 Conclusion and future work

Within a single A100 budget, R2-Dreamer exhibits clean, sizable positive transfer on a pixel-based `walker_walk` $\rightarrow$ `walker_run` shift—large warm start, several-fold mid-training sample efficiency, a +179 final advantage at equal budget—together with a distinct, better-structured latent representation in the fine-tuned model. The most valuable next steps are: (1) **2–3 additional seeds** for the target pair to put error bars on the advantage; (2) **extended budgets** for scratch (and fine-tune) to separate "head start" from "higher ceiling"; (3) a **representation-loss ablation** (`dreamer` vs. `r2dreamer` vs. `infonce` vs. `dreamerpro`) to attribute the effect; and (4) **harder shifts** (e.g. Meta-World task transfer, or proprio$\rightarrow$vision) to test the generality of the transfer.

Reproduction

Open `notebooks/01_r2dreamer_foundation.ipynb` on a Colab A100 with a Python 3.11 runtime; the default `a100_vision` preset reproduces the runs above (set `R2_PRESET=t4_proprio` for the smaller proprioceptive variant). After the smoke test, run the source, fine-tune, and scratch cells. Results, rollout videos, and the latent analysis are regenerated by `notebooks/06_r2dreamer_results.ipynb`, from which every figure and number in this report is taken.

References

[1] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. *Mastering Diverse Domains through World Models*. arXiv:2301.04104, 2023.

[2] D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba. *Mastering Atari with Discrete World Models*. ICLR, 2021. arXiv:2010.02193.

[3] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. *Dream to Control: Learning Behaviors by Latent Imagination*. ICLR, 2020. arXiv:1912.01603.

[4] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. *Learning Latent Dynamics for Planning from Pixels*. ICML, 2019. arXiv:1811.04551.

[5] F. Deng, I. Jang, and S. Ahn. *DreamerPro: Reconstruction-Free Model-Based Reinforcement Learning with Prototypical Representations*. ICML, 2022. arXiv:2110.14565.

[6] J. Zbontar, L. Jing, I. Misra, Y. LeCun, and S. Deny. *Barlow Twins: Self-Supervised Learning via Redundancy Reduction*. ICML, 2021. arXiv:2103.03230.

[7] A. van den Oord, Y. Li, and O. Vinyals. *Representation Learning with Contrastive Predictive Coding*. arXiv:1807.03748, 2018.

[8] Y. Tassa, Y. Doron, A. Muldal, T. Erez, Y. Li, D. de Las Casas, D. Budden, A. Abdolmaleki, J. Merel, A. Lefrancq, T. Lillicrap, and M. Riedmiller. *DeepMind Control Suite*. arXiv:1801.00690, 2018.

[9] S. Tunyasuvunakool, A. Muldal, Y. Doron, S. Liu, S. Bohez, J. Merel, T. Erez, T. Lillicrap, N. Heess, and Y. Tassa. *dm_control: Software and Tasks for Continuous Control*. Software Impacts, 6:100022, 2020.

[10] E. Todorov, T. Erez, and Y. Tassa. *MuJoCo: A Physics Engine for Model-Based Control*. IROS, 2012.

[11] NM512. *r2dreamer*. GitHub repository, https://github.com/NM512/r2dreamer.

[12] NM512. *dreamerv3-torch: PyTorch Implementation of DreamerV3*. GitHub repository, https://github.com/NM512/dreamerv3-torch.